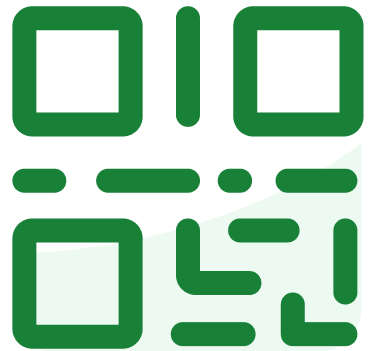


Scalable AI

Self-attentive architectures

Lorenzo Baraldi
Marta Lovino

slido



**Join at slido.com
#41689494**

ⓘ Start presenting to display the joining instructions on this slide.

slido



What do you remember from previous courses about the attention operator?

ⓘ Start presenting to display the poll results on this slide.

MATRIX MULTIPLICATION

**THE NUMBER OF COLUMNS IN THE FIRST MATRIX
HAS TO EQUAL THE NUMBER OF ROWS IN THE
SECOND ONE**

A step back in linear algebra

If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is an $n \times p$ matrix,

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{pmatrix}$$

the *matrix product* $\mathbf{C} = \mathbf{AB}$

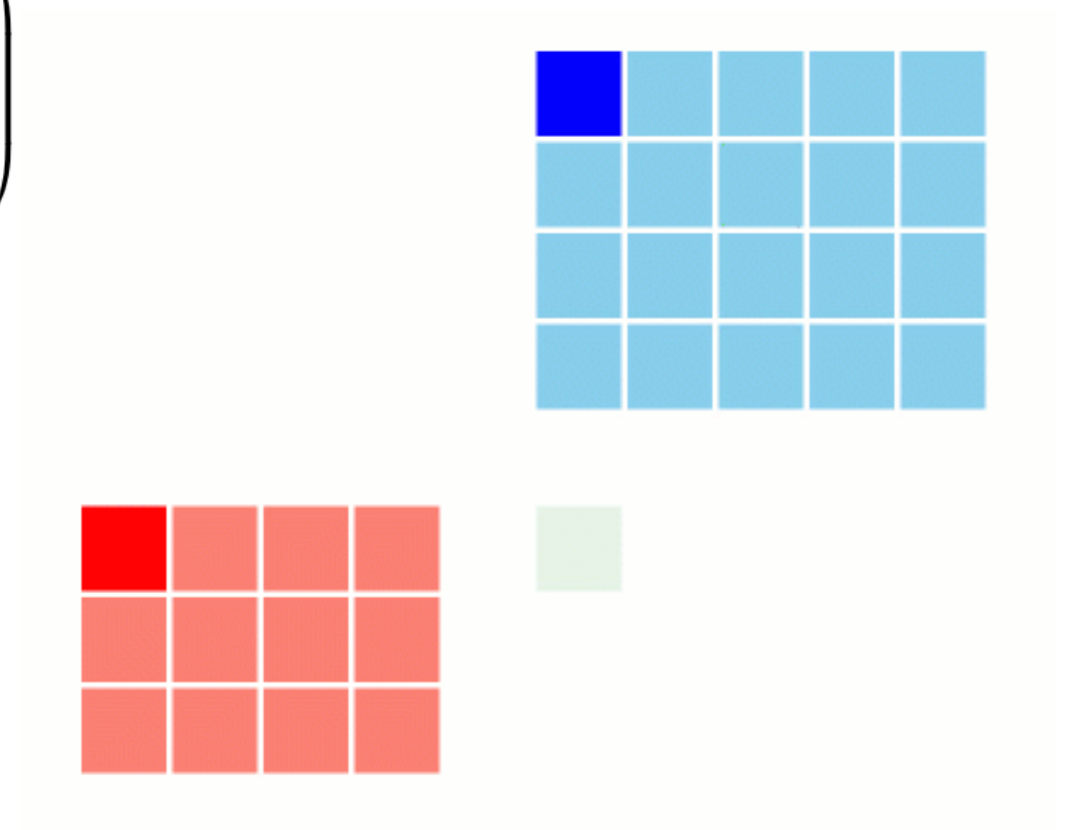
is defined to be the $m \times p$

$$\mathbf{C} = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{pmatrix}$$

such that

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj} = \sum_{k=1}^n a_{ik}b_{kj},$$

for $i = 1, \dots, m$ and $j = 1, \dots, p$.



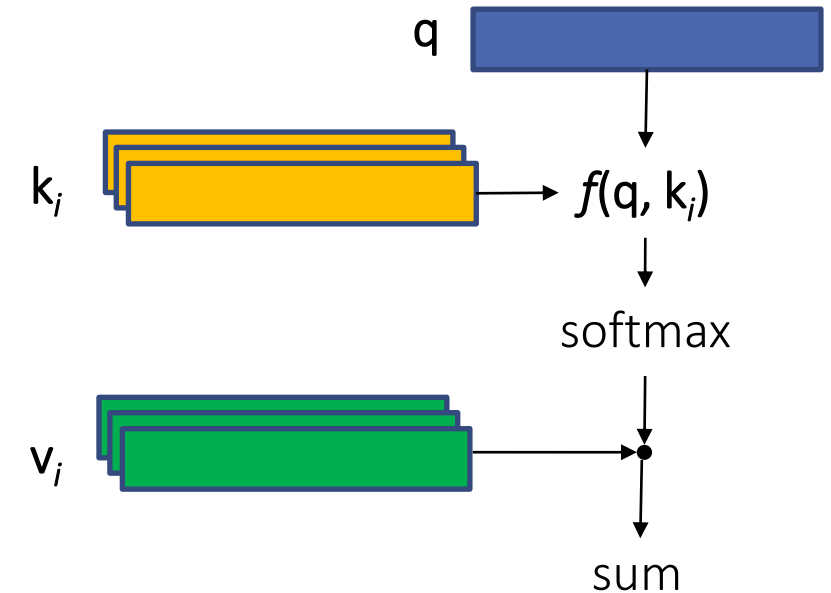
A new operator

Attention

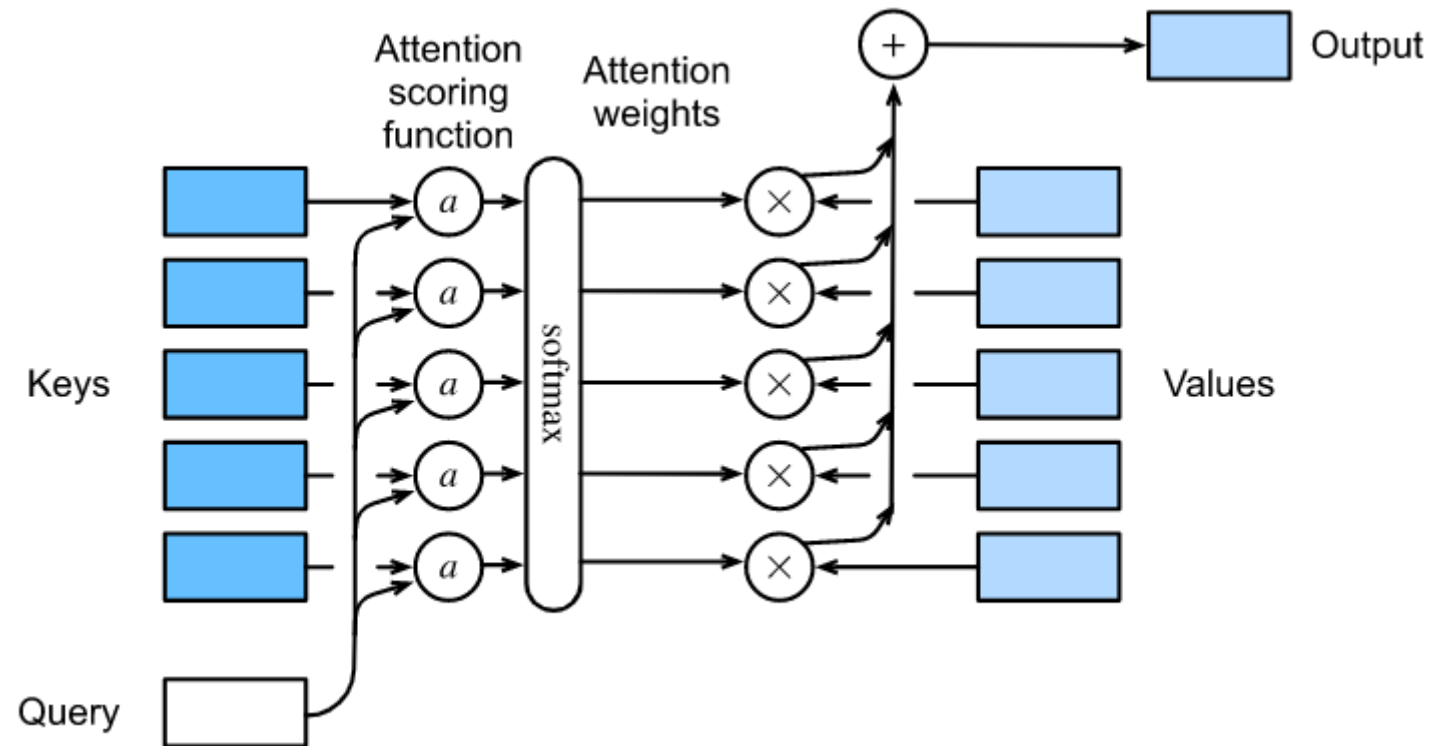
Provides a way to focus on part of an input set.

Given a query and pairs of keys and values,

- Compute similarities between queries and keys
- Normalizes similarities via softmax to obtain attention scores
- Multiplies values by the scores



Attention



Mathematically, suppose that we have a query $\mathbf{q} \in \mathbb{R}^q$ and m key-value pairs $(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)$, where any $\mathbf{k}_i \in \mathbb{R}^k$ and any $\mathbf{v}_i \in \mathbb{R}^v$. The attention pooling f is instantiated as a weighted sum of the values:

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i \in \mathbb{R}^v, \quad (10.3.1)$$

where the attention weight (scalar) for the query $\mathbf{q}$ and key $\mathbf{k}_i$ is computed by the softmax operation of an attention scoring function a that maps two vectors to a scalar:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))} \in \mathbb{R}. \quad (10.3.2)$$

Attention

Attention

Provides a way to focus on part of an input set.

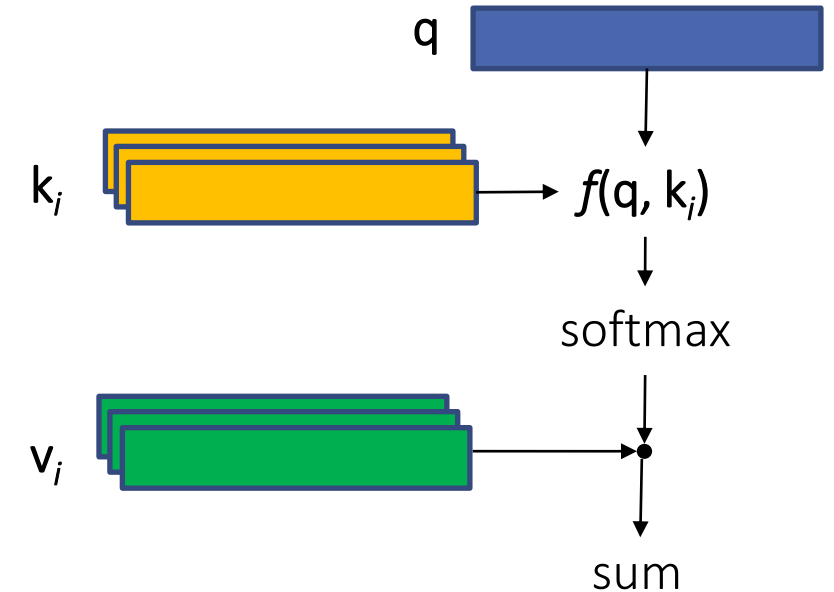
Given a query and pairs of keys and values,

- Compute similarities between queries and keys
- Normalizes similarities via softmax to obtain attention scores
- Multiplies values by the scores

Similarity function

- Additive attention, e.g. $w^T \tanh(w_q q + w_k k)$
- Dot-product attention

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T \mathbf{k} / \sqrt{d}$$



Dot-product attention

```
class DotProductAttention(nn.Module):
    """Scaled dot product attention."""
    def __init__(self, **kwargs):
        super(DotProductAttention, self).__init__(**kwargs)

    # Shape of `queries`: (`batch_size`, no. of queries, `d`)
    # Shape of `keys`: (`batch_size`, no. of key-value pairs, `d`)
    # Shape of `values`: (`batch_size`, no. of key-value pairs, value
    # dimension)
    # Shape of `valid_lens`: (`batch_size`,) or (`batch_size`, no. of queries)
    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        # Use `transpose` to swap the last two dimensions of `keys`
        scores = torch.bmm(queries, keys.transpose(1, 2)) / math.sqrt(d)
        self.attention_weights = torch.softmax(scores)
        return torch.bmm(self.attention_weights, values)
```

Convolution vs Self-attention

Self Attention

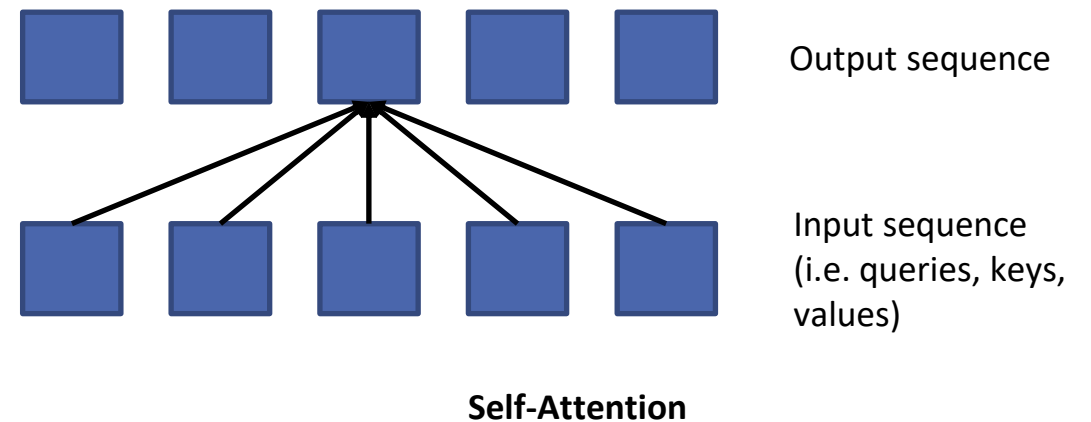
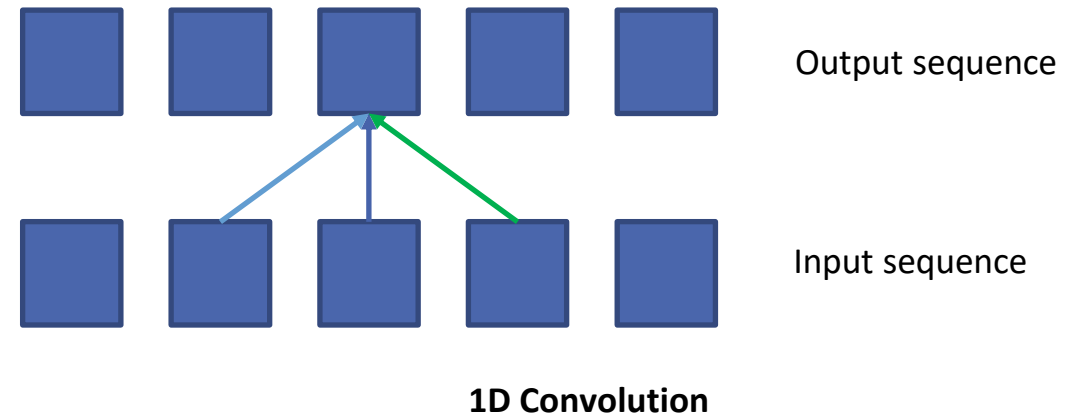
“Refine” each element of the sequence by treating it as **query**, and the whole sequence as keys and values.

Actually: queries, keys and values are three different linear projections of each element of the input sequence.

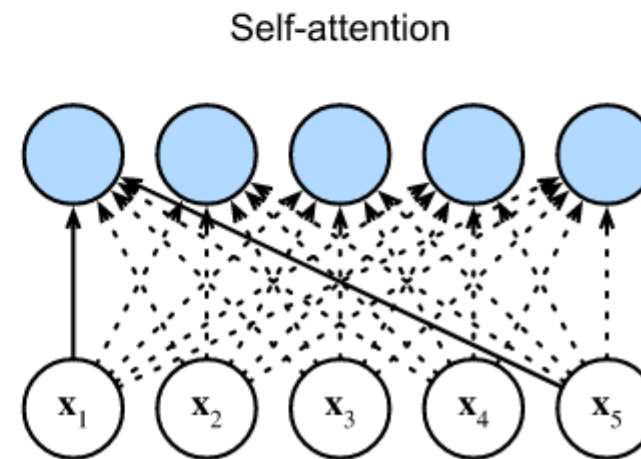
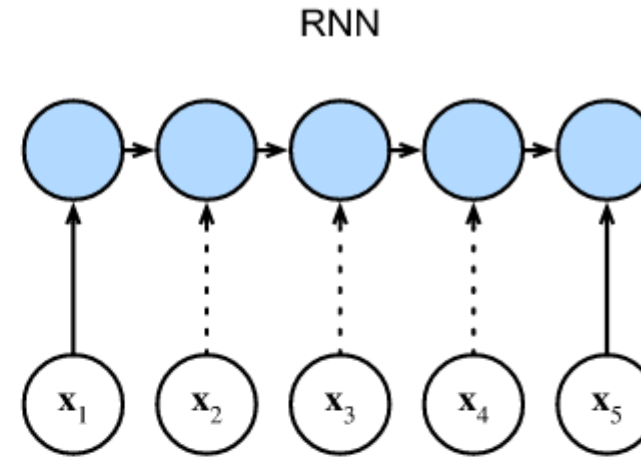
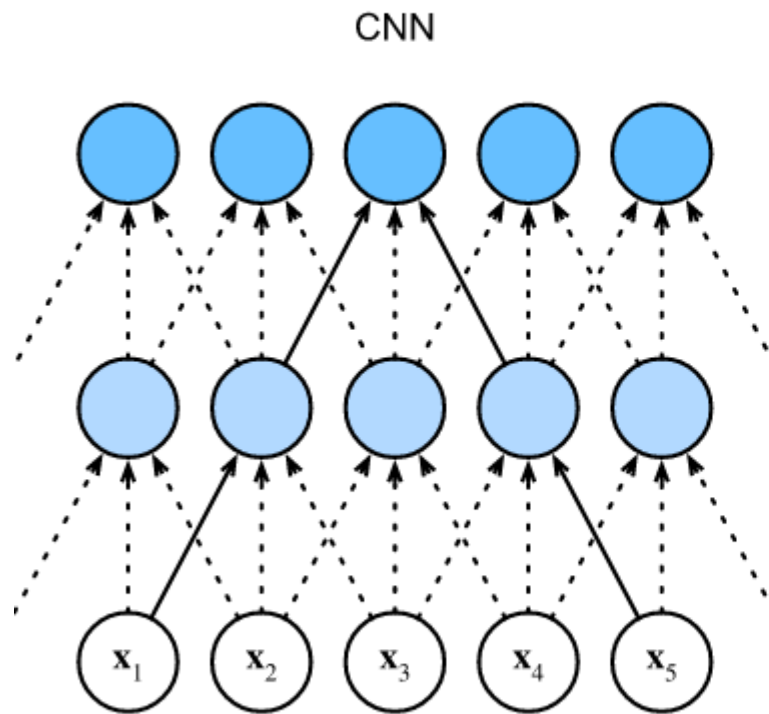
Receptive field is infinite!

Constant path length between two different positions

Trivial to parallelize during training!



CNNs vs RNNs vs Self-attention



Self-attention

- Given the input sequence, extract queries, keys and values from each element by applying a linear transformation (i.e. a fully connected layer). The dimensionality of queries and keys need to match.

From a sequence with length T , we obtain:

- Q : matrix of queries, (T, d_k)
 - K : matrix of keys, (T, d_k)
 - V : matrix of values, (T, d_v)
- Apply scaled dot-product attention as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

slido

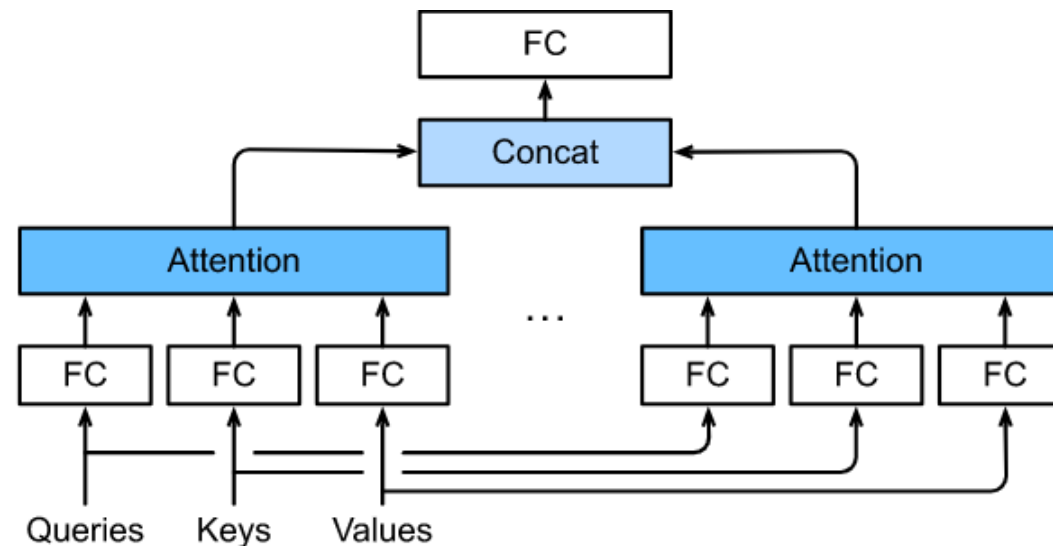


Audience Q&A Session

① Start presenting to display the audience questions on this slide.

Multi-head attention

- In practice, given the same set of queries, keys, and values we may want our model to combine knowledge from different behaviors of the same attention mechanism, such as capturing dependencies of various ranges in a sequence.
- To this end, instead of performing a single attention pooling, queries, keys, and values can be transformed with h **independently learned linear projections**. Then these h projected queries, keys, and values are fed into attention pooling **in parallel**. In the end, h attention pooling outputs are concatenated and transformed with another learned linear projection to produce the final output.



Multi-head Self-attention

- Linearly project the input sequence h times with different weights, instead of doing this only once.
- To From a sequence with length T , we obtain:
 - Q : matrix of queries, (h, T, d_k)
 - K : matrix of keys, (h, T, d_k)
 - V : matrix of values, (h, T, d_v)
- Apply scaled dot-product attention over each “head” (i.e. over each element of axis 1)
- Concatenate the result and project back to a lower dimensionality

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(Q[i], K[i], V[i])$

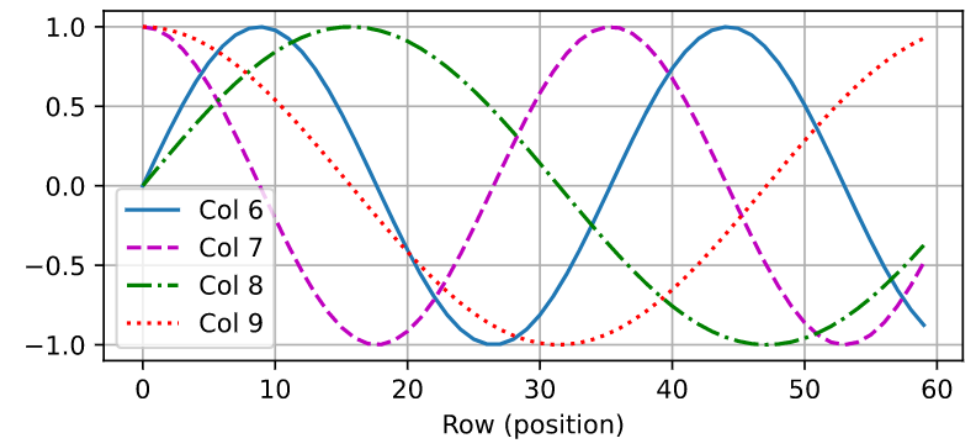
- Can be done in parallel, with batched matrix multiplication.

Positional encoding

- Self-attention is permutation invariant
 - Given a query, if we change the order of keys and values, result does not change.
 - Ok for encoding sets. Not for sequences or images....☹
- To use **order information**, we can inject absolute or relative positional information by adding positional encoding to the input representations.
- Positional encodings can be learned (simple nn.Parameter) or fixed. In the original Transformer, they were defined as sinusoids. With this, attention can capture both absolute and relative positional information (i.e. distances between items!)

Suppose that the input representation $\mathbf{X} \in \mathbb{R}^{n \times d}$ contains the d -dimensional embeddings for n tokens of a sequence. The positional encoding outputs $\mathbf{X} + \mathbf{P}$ using a positional embedding matrix $\mathbf{P} \in \mathbb{R}^{n \times d}$ of the same shape, whose element on the i^{th} row and the $(2j)^{\text{th}}$ or the $(2j + 1)^{\text{th}}$ column is

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right),$$
$$p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$



A Self-attentive language model for translation

Encoder

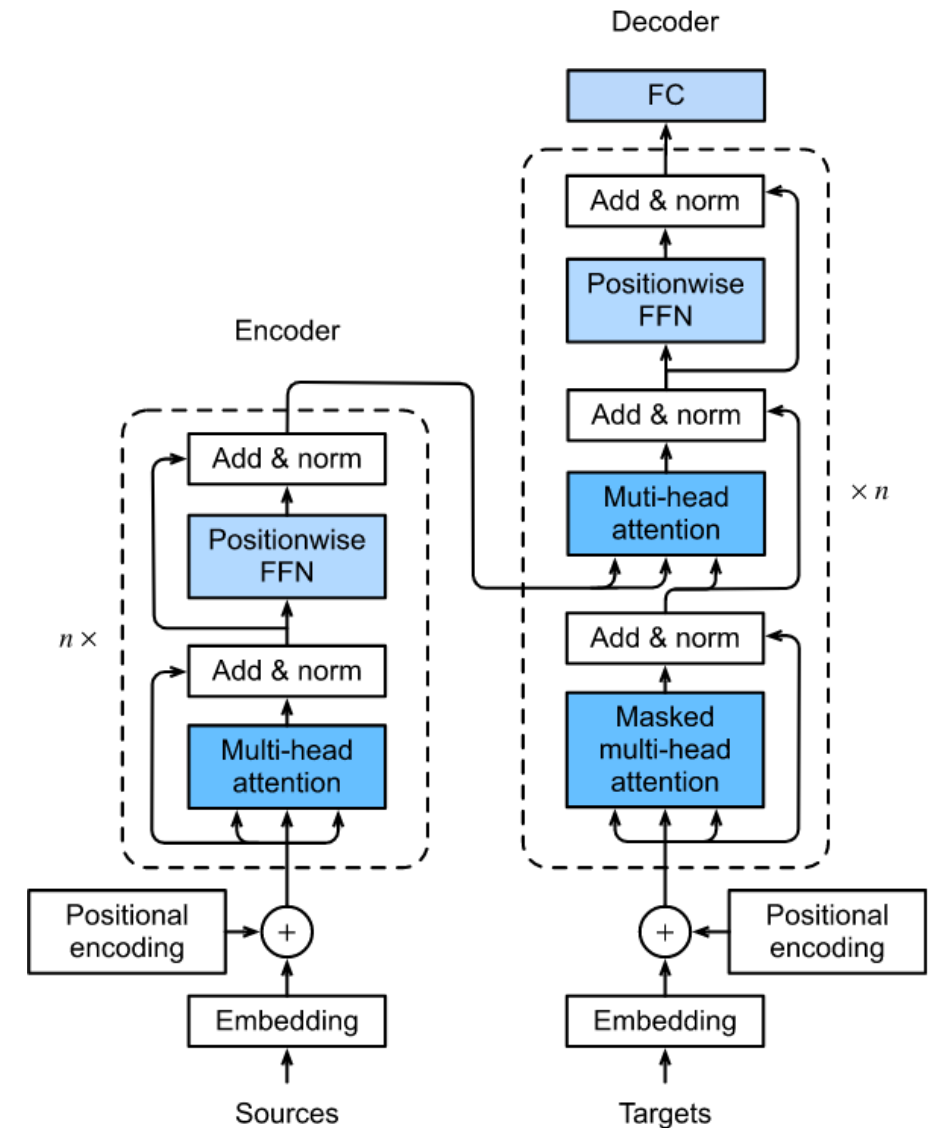
Uses self-attention on its input

Multiple attention layers stacked together (with add+norm) and feed-forward layers (linear layers applied timewise).

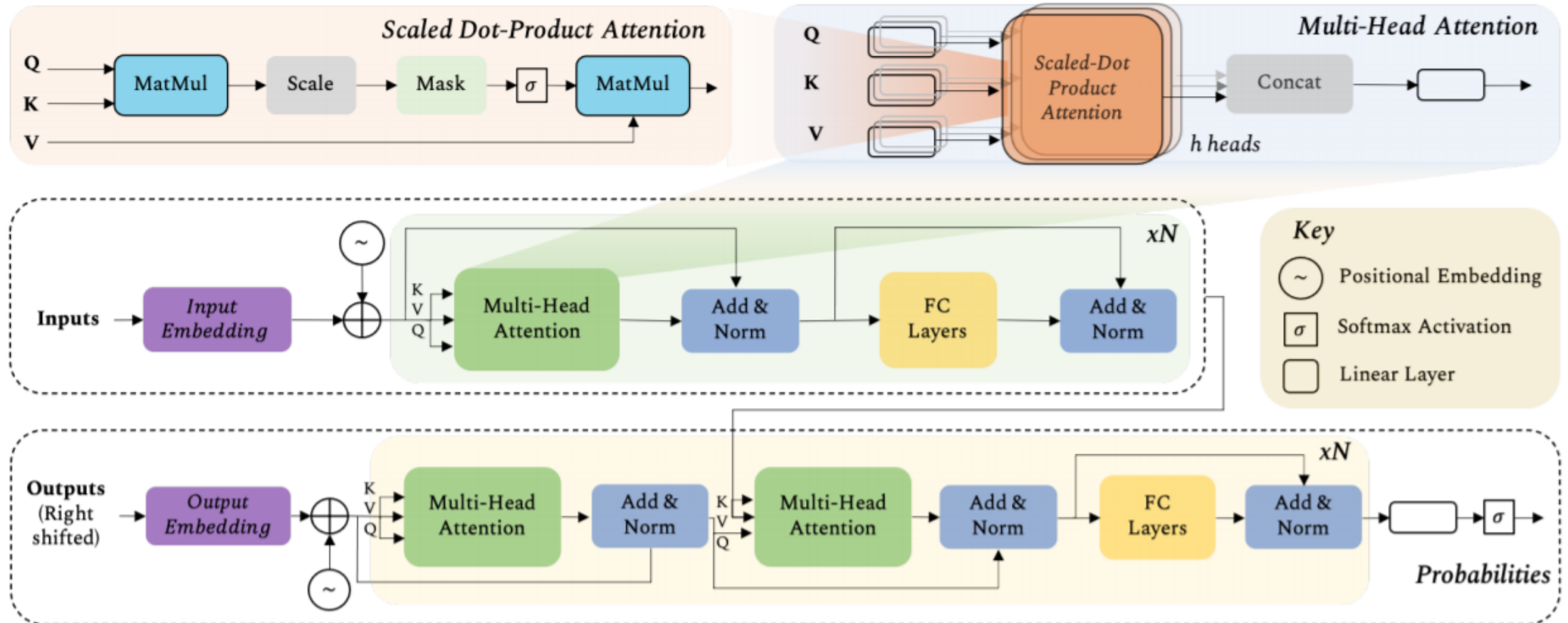
Decoder

Self-attention on words

Cross-attention on encoder outputs: use decoder sequences as queries, encoder outputs as key/values.



Architecture of the Transformer





Audience Q&A Session

① Start presenting to display the audience questions on this slide.

Vision Transformer (ViT)

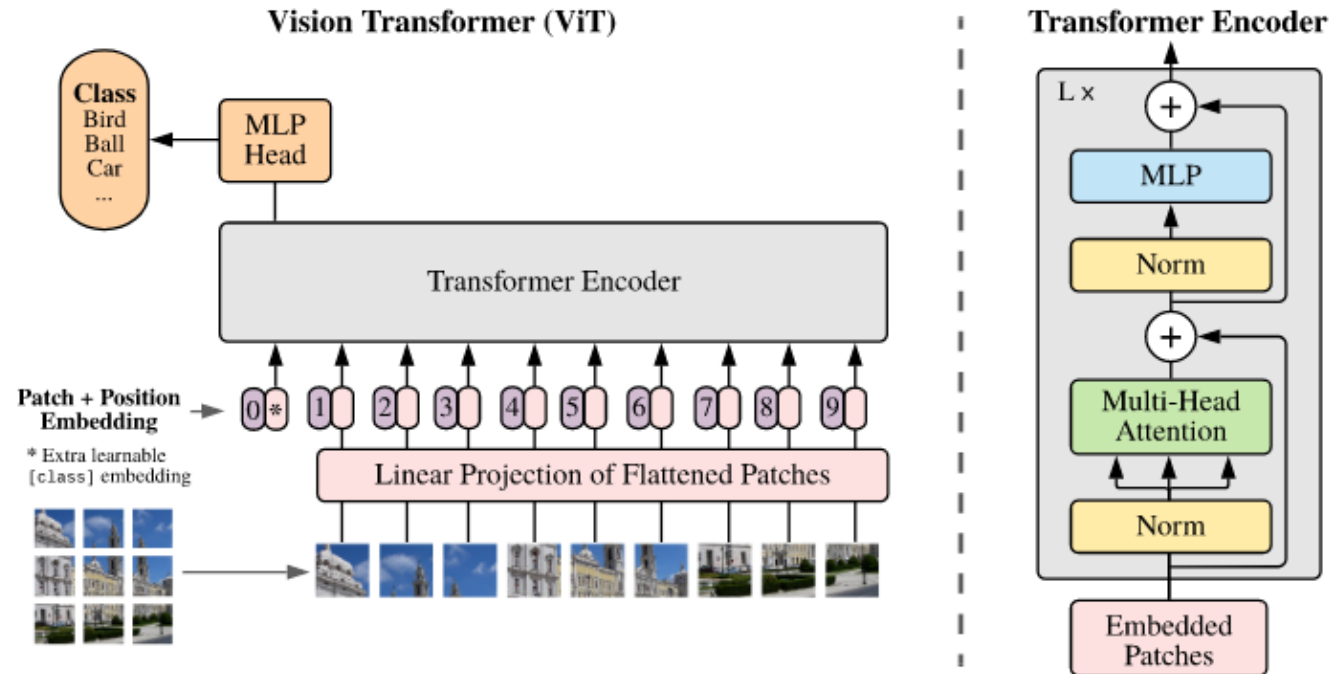


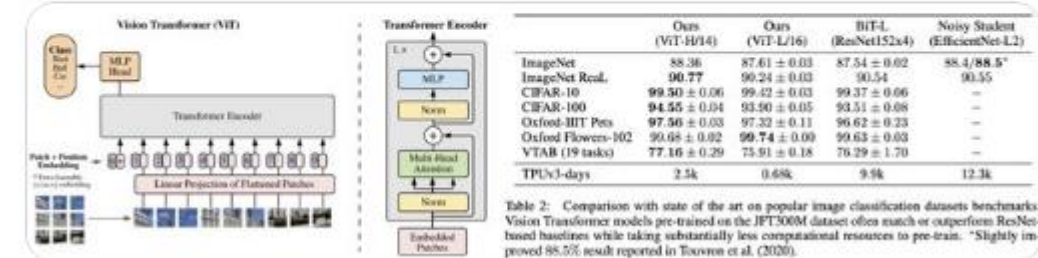
Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by [Vaswani et al. \(2017\)](#).

Vision Transformer (ViT)



Andrej Karpathy @karpathy · Oct 3

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale openreview.net/forum?id=YicbF... v cool. Further steps towards deprecating ConvNets with Transformers. Loving the increasing convergence of Vision/NLP and the much more efficient/flexible class of architectures.



26

522

1.9K



Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Data-efficient Vision Transformers

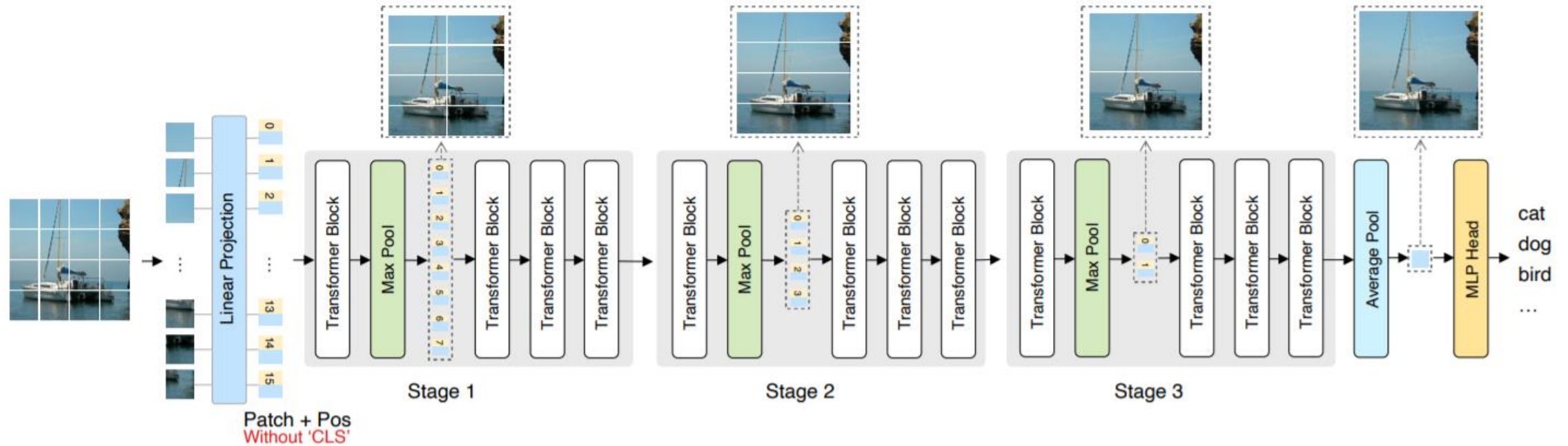
- ViT was trained on **JFT-300M**, a large **private** dataset with 300 millions images. The paper concluded that transformers “*do not generalize well when trained on insufficient amounts of data*”.
- Data-efficient VT: train a vision transformer on a single 8-GPU node in two to three days (53 hours of pre-training, and optionally 20 hours of fine-tuning) that is competitive with convnets having a similar number of parameters and efficiency. **It uses Imagenet as the sole training set.**
- Key: Knowledge Distillation from CNNs! 😊

Training data-efficient image transformers & distillation through attention

Hugo Touvron^{*,†} Matthieu Cord[†] Matthijs Douze^{*}
Francisco Massa^{*} Alexandre Sablayrolles^{*} Hervé Jégou^{*}

^{*}Facebook AI [†]Sorbonne University

What about pooling?



Object detection

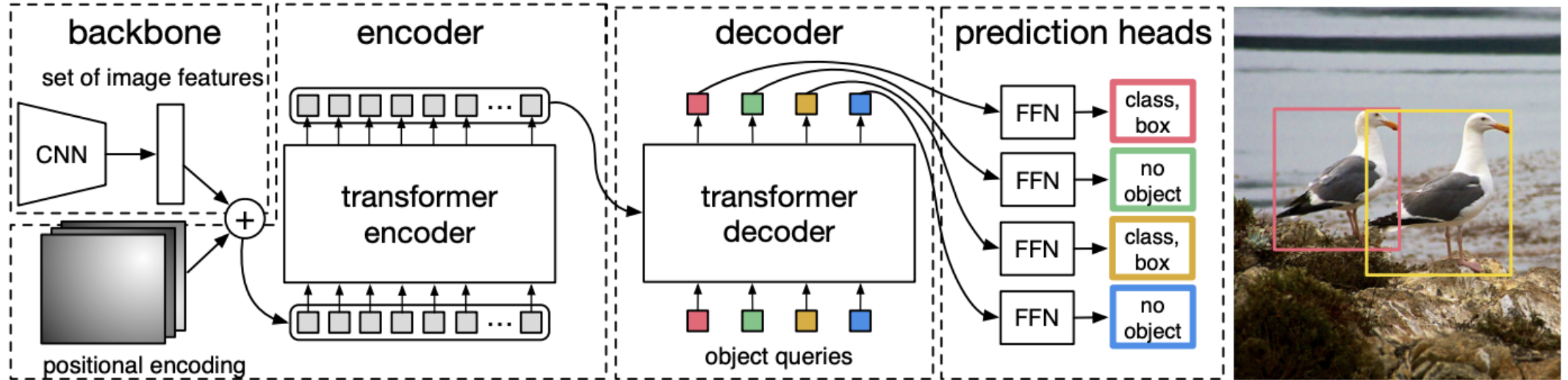


Fig. 2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “no object” class.

Segmentation

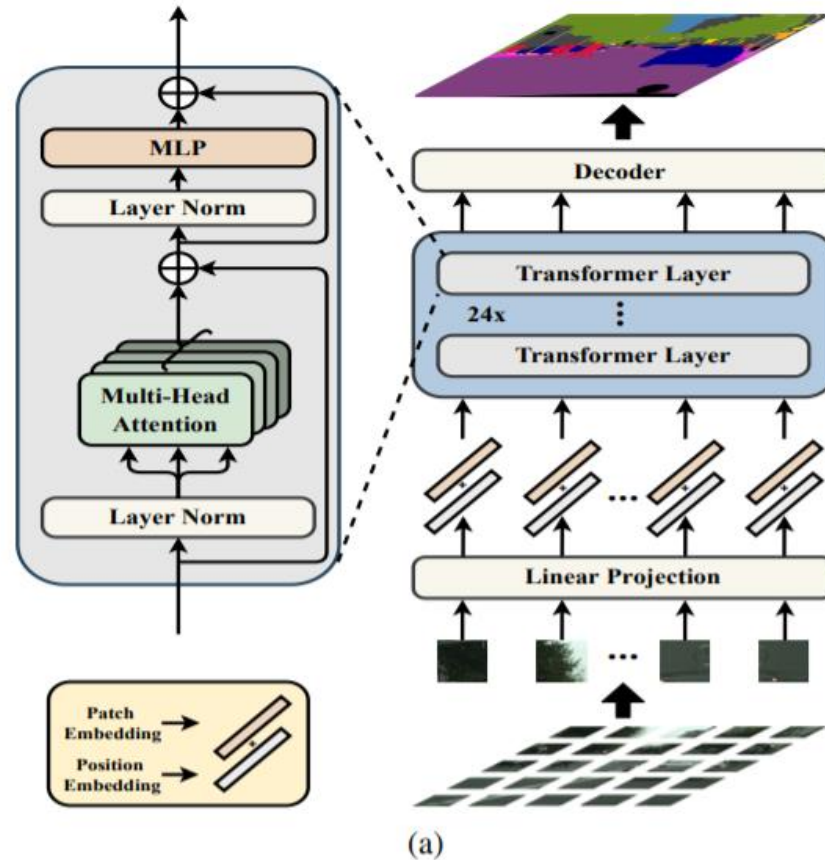


Figure 1. **Schematic illustration of the proposed SEgmentation TRansformer (SETR)** (a). We first split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. To perform pixel-wise segmentation, we introduce different decoder designs: (b) progressive upsampling (resulting in a variant called SETR-PUP); and (c) multi-level feature aggregation (a variant called SETR-MLA).



Audience Q&A Session

① Start presenting to display the audience questions on this slide.